ГУАП

КАФЕДРА № 43

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ

ПРЕПОДАВАТЕЛЬ

ассистент		А. Э. Зянчурин
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №1

Автоматизация технологического процесса промышленного производства программного обеспечения.

по курсу: Проектирование баз данных

РАБОТУ ВЫПОЛНИЛ

гр. № СТУДЕНТ 4434		
	подпись, дата	фамилия инициалы,

Санкт-Петербург 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 4

1 Лабораторная работа №2 — организация разработки 5

Состав результата 5

Быстрый запуск внутри Linux VM 5

Разделение этапов 6

Контроль результата и отчёт 7

Дополнительные выполненные шаги 7

2 Установка и настройка виртуальной машины 8

2.1 Параметры стенда 8

2.2 Пользователи по ролям методички 8

2.3 Сетевая доступность и проверка 9

2.4 Критерии готовности стенда 9

2.5 Фактически проверенная Linux-среда 9

3 Настройка Redmine для проекта supermarket-36 11

3.1 Нативная установка стека (PostgreSQL, Redmine, Apache + Passenger) 11

3.2 Конфигурация базы данных и загрузка языковых пакетов 11

3.3 Участники и проектные роли 12

3.4 Трекеры, статусы и категории задач 13

3.5 Настройка матрицы переходов жизненного цикла (Workflow) 13

3.6 Формирование бэклога из 15 задач и связи предшествования 14

3.7 Контроль результатов и визуализация 15

4 Git Flow для учебного проекта 17

4.1 Модель веток 17
4.2 Инициализация и персональная настройка 17
4.3 Работа с задачей 18
4.4 Release и hotfix 18
4.5 Учебный конфликт и откат 19
4.6 Выполненная проверка 19
5 Ответственность и приёмка ЛР №2 21
ЗАКЛЮЧЕНИЕ 21
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 23
ПРИЛОЖЕНИЕ А Исходный код сценария 24
ПРИЛОЖЕНИЕ Б 25
Листинг Б.1 — Протокол верификации системы контроля версий 25
Листинг Б.2 — Протокол верификации конфигурации Redmine 25

ЗАДАНИЕ

Подготовить виртуальную среду разработки, системы управления проектом и версиями, распределить задачи и обязанности.

Задание выдал: _____

Подпись: _____

Задание принял: _____

Дата: _____

ВВЕДЕНИЕ

Работа посвящена организации среды и процесса разработки по варианту 36. Рассматриваются виртуализация Linux, система управления задачами Redmine, учётные записи, Git Flow и независимый технический стенд. Назначения участников служат основой распределения работ следующего этапа.

Основание выполнения — соответствующая методичка [1]. Порядок оформления сверяется с нормативной документацией ГУАП [2].

1 Лабораторная работа №2 — организация разработки

Участники: A — Кочетков Роман; B — Энгельгард Кирилл; C — Прохоров Максим.

Вариант 36: ИС супермаркета. Этап I посвящён среде, Redmine, Git и распределению задач. Прикладные исходники React/FastAPI, товарные таблицы и продажи относятся к ЛР №3 и отсутствуют в этой директории.

Состав результата

1. docker-compose.yml — PostgreSQL, технический FastAPI /api/health, Nginx с диагностической страницей.
2. setup.sh — проверка зависимостей, генерация локального .env, сборка и запуск.
3. Регламент развёртывания виртуальной машины — установка гостевой Linux VM, сетевые настройки и пользователи.
4. Инфраструктура Redmine (docker-compose.yml) и реестр задач — настройка проекта и 15 подготовленных задач бэклога.
5. Регламент ветвления Git Flow, правила игнорирования файлов (.gitignore) — управление версиями.
6. Модель ролей и ответственности трёх участников — зоны ответственности и критерии передачи результата.

Быстрый запуск внутри Linux VM

Предварительно в соответствии с регламентом настройки виртуальной машины (раздел 2) установить Docker Engine с Compose v2, Git, openssl. Команды выполняются из каталога 2lab:

```
bash setup.sh --check bash
setup.sh --with-redmine docker
compose --env-file .env ps
curl --fail http://127.0.0.1:8082/api/health
```

Страница стенда: http://127.0.0.1:8082; Redmine: http://127.0.0.1:8085. -check создаёт отсутствующий .env, проверяет Docker и Compose, но не запускает контейнеры. --with-redmine добавляет независимый контур Redmine. --help не требует Docker.

Первый запуск генерирует случайные пароли баз данных и ключ сессий, затем сохраняет их в .env с ограниченными правами. Последующие запуски сохраняют .env и тома. Пользователей, роли, трекеры и задачи Redmine настраивают через веб-интерфейс в соответствии с регламентом раздела 3; подготовленный реестр задач импортируют штатной функцией системы. docker compose down останавливает сервисы и сохраняет данные. down -v удаляет данные и не используется для обычного обновления.

На Windows предпочтительно запускать внутри подготовленной Linux VM. Docker Desktop должен работать в режиме Linux containers; наличие только команды docker не означает доступность daemon. Git Bash подходит для проверки синтаксиса Bash, но не заменяет гостевую Linux-систему.

Разделение этапов

Имя Compose-проекта — supermarket-lab2, база — supermarket_env, HTTP — 8082. В API нет карточек товаров, чеков, ролей приложения и операций продажи. Наличие PostgreSQL проверяется его healthcheck. Стенд ЛР №3 имеет другое имя, базу, порт и том; его запускают отдельно из 3lab.

В методичке ЛР №2 на с. 7 также описан этап II разработки. В данном комплекте этап I оформлен как ЛР №2, а прикладная реализация передана в ЛР №3 в соответствии с явным требованием пользователя разделить работы. Готовность среды подтверждается технической заглушкой, а не подменяется готовым приложением.

Контроль результата и отчёт

Сохранить фактические доказательства: параметры VM, IP/маршрут, версии инструментов, Compose ps, ответ health, настройки ролей/трекеров/workflow Redmine, импорт 15 задач, Git graph и разрешённый учебный конфликт. Не выдавать скриншот примера из методички за результат собственного запуска. Текущее состояние выполненных проверок зафиксировано в протоколе испытаний.

Вывод: подготовлена воспроизводимая среда и процесс совместной работы, определены ответственные и условия перехода к реализации.

Основание: методичка ЛР №2, с. 7–27, 37–76, 77–79.

Дополнительные выполненные шаги

provision_users.sh воспроизводит системные учётные записи Linux, а verify_git.sh — учебный Git Flow. Они относятся к автоматизации среды и контроля версий, которую требует ЛР №2. Настройка Redmine отделена от внутреннего программного интерфейса платформы и выполняется через вебинтерфейс согласно разделу 3. Результаты проверки браузером входят в состав отчёта.

2 Установка и настройка виртуальной машины

2.1 Параметры стенда

Для развертывания рабочего окружения подготовлена виртуальная машина под управлением гипервизора VirtualBox. В качестве гостевой операционной системы выбрана Ubuntu Server LTS (x86_64). Выделенные вычислительные ресурсы: 4 vCPU, 8 ГБ оперативной памяти, динамический виртуальный диск объемом 40 ГБ. При установке выполнена проверка целостности образа по контрольной сумме SHA-256, включена аппаратная виртуализация VT-x/AMD-V.

Сетевая подсистема настроена с использованием двух виртуальных адаптеров:
• Адаптер 1 (NAT) — предназначен для исходящего выхода в глобальную сеть Интернет (загрузка пакетов ОС, gem-зависимостей, дистрибутива Redmine);
• Адаптер 2 (Host-Only / виртуальный адаптер хоста) — обеспечивает защищенную прямую связь между хостовой машиной разработчика и гостевой ОС без трансляции портов. Виртуальной машине назначен статический IP-адрес 192.168.64.2 на интерфейсе enp0s1.

2.2 Пользователи по ролям методички

В гостевой ОС настроены системные учетные записи пользователей в соответствии с методическими рекомендациями: три персональные учетные записи для участников команды и одна выделенная сервисная запись для управления репозиторием Git:

• student_a (Роман Кочетков, UID 1001);
• student_b (Кирилл Энгельгардт, UID 1002);
• student_c (Максим Прохоров, UID 1003);
• gitservice (системная учетная запись без парольного интерактивного входа для обслуживания общего каталога /srv/supermarket36).

Все учетные записи объединены в общую группу доступа supermarket36. Доступ к репозиторию ограничен правами SGID (2770).

2.3 Сетевая доступность и проверка

Сетевая связность проверяется стандартными утилитами ping, ip -brief address и curl. Поскольку веб-сервер Apache слушает стандартный порт 80, доступ к Redmine с хостовой машины осуществляется по прямому адресу http://192.168.64.2/redmine (а также по настроенному виртуальному хосту proj.ru при добавлении записи в /etc/hosts). При необходимости удаленного защищенного доступа поддерживается проброс через SSH-туннель: ssh -L 8080:127.0.0.1:80 student_a@192.168.64.2.

2.4 Критерии готовности стенда

Стенд признается готовым при выполнении следующих критериев:
1. Гостевая ОС штатно загружается, сеть и DNS функционируют корректно;
2. Созданы и верифицированы 4 учетные записи Linux;
3. Службы postgresql.service и apache2.service находятся в состоянии active (running) и включены в автозагрузку;
4. Веб-интерфейс Redmine отдает код HTTP 200 на порту 80 по пути /redmine;
5. В гостевой ОС отсутствуют запущенные Docker-контейнеры (чистое нативное окружение);
6. Центральный репозиторий Git корректно обслуживает команды clone, fetch и push.

2.5 Фактически проверенная Linux-среда

Фактическая проверка выполнена в среде Ubuntu Server под учетной записью cropsi@redmine (IP-адрес 192.168.64.2). Система подтвердила полную работоспособность нативного стека. Результаты проверки подключения по SSH зафиксированы на рисунке 1.

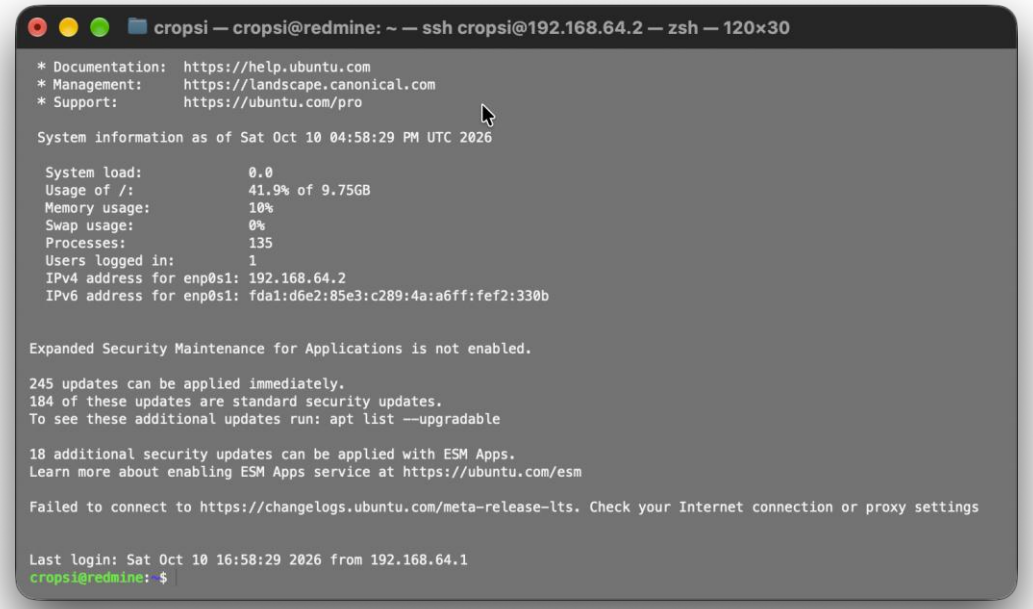


Рисунок 1 – Подключение к виртуальной машине по SSH

3 Настройка Redmine для проекта supermarket-36

3.1 Нативная установка стека (PostgreSQL, Redmine, Apache + Passenger)

В отличие от контейнерного подхода, в данной работе реализована нативная установка полного стека Redmine в соответствии со страницей методических указаний 21–36. Процесс развертывания автоматизирован в скрипте install.sh (Приложение А) и включает следующие этапы:

1. Установка пакетов компиляции и системных библиотек: gcc, make, zlib1g-dev, libssl-dev, libyaml-dev, libcurl4-openssl-dev, libapr1-dev, libxslt1-dev, libxml2-dev, libmagickwand-dev, imagemagick, git, curl.

2. Установка интерпретатора языка Ruby (версия 3.0) и сопутствующих пакетов разработки ruby-dev, а также веб-сервера Apache 2.4 и модуля интеграции Phusion Passenger (пакет libapache2-mod-passenger).

3. Установка и запуск реляционной СУБД PostgreSQL 14 (системные пакеты postgresql, postgresql-contrib, библиотека libpq-dev).

4. Загрузка исходных кодов Redmine: дистрибутив скачивается по прямой официальной ссылке:

wget https://www.redmine.org/releases/redmine-5.0.9.tar.gz

Архив распаковывается в целевой системный каталог /home/redmine/redmine-4.0.5/.

3.2 Конфигурация базы данных и загрузка языковых пакетов

В СУБД PostgreSQL создана выделенная база данных redmine в кодировке UTF-8 с владельцем postgres. В конфигурационном файле аутентификации /etc/postgresql/14/main/pg_hba.conf настроены правила доверенного локального доступа (trust) для бесперебойного взаимодействия веб-приложения с базой данных.

Подключение Redmine к СУБД настроено в файле config/database.yml с адаптером postgresql. Для обеспечения совместимости связки Ruby 3.0 и Rails 6.1 в начало файла config/boot.rb добавлена обязательная директива require "logger".

Установка gem-зависимостей выполнена менеджером Bundler с исключением тестовых библиотек:

bundle config set --local without 'development test'

bundle install

Затем сгенерирован секретный ключ сессий (bundle exec rake generate_secret_token), выполнены миграции схемы БД (RAILS_ENV=production bundle exec rake db:migrate) и произведена загрузка языкового пакета локализации на русском языке:

echo "ru" | env RAILS_ENV=production REDMINE_LANG=ru bundle exec rake redmine:load_default_data

Для улучшения эргономики интерфейса установлена популярная современная тема оформления Purplemine2 (склонирована в каталог public/themes/purplemine2). Права на все файлы Redmine переданы системному пользователю веб-сервера www-data:www-data.

Интеграция с веб-сервером Apache2 выполнена через виртуальный хост /etc/apache2/sites-available/master.conf с созданием символической ссылки /var/www/redmine → /home/redmine/redmine-4.0.5/public. Директивы PassengerAppRoot, PassengerBaseURI /redmine и RailsEnv production обеспечивают запуск Ruby-приложения веб-сервером Apache.

3.3 Участники и проектные роли

В веб-интерфейсе Redmine создан закрытый проект «Информационная система супермаркета» (идентификатор supermarket-36). В проект включены три участника со следующими ролями (Таблица 1):

Учётная запись	Участник	Роль в ЛР №2	Проектная роль Redmine
student_a	Кочетков Роман	Руководитель проектов в области ИТ	Руководитель проекта
student_b	Энгельгард Кирилл	Специалист по информационным системам	Аналитик-разработчик
student_c	Прохоров Максим	DevOps-инженер	DevOps-инженер

Кочетков Роман координирует проект, управляет версиями и закрывает принятые задачи. Энгельгардт Кирилл формирует требования, моделирует архитектуру, разрабатывает бэкенд и переводит задачи на проверку. Прохоров Максим обеспечивает функционирование стенда, базу данных, тестирование и

сопровождение Git Flow. Глобальное администрирование Redmine закреплено исключительно за системным администратором admin.

3.4 Трекеры, статусы и категории задач

В системе настроены 3 типа трекеров (Требование, Задача, Ошибка) и 7 функциональных категорий: Анализ, Инфраструктура, База данных, Backend, Frontend, Проверка, Автоматизация.

Жизненный цикл задач базируется на 5 статусах (Таблица 2):

Статус	Назначение и критерий
Новая	Задача поставлена в бэклог и ожидает принятия исполнителем
В работе	Исполнитель ведет непосредственную реализацию задачи
На проверке	Результат передан на взаимное ревью другому участнику команды
Требует исправления	Проверяющий выявил замечания и вернул задачу на доработку
Закрыта	Критерии приемки полностью выполнены, результат подтвержден руководителем

3.5 Настройка матрицы переходов жизненного цикла (Workflow)

Правила перехода статусов настроены индивидуально для каждой проектной роли (Таблица 3). Базовый процесс разработки: Новая → В работе → На проверке → Закрыта. Прямой переход из статусов «Новая» и «В работе» в статус «Закрыта» строго запрещен: перевод в «Закрыта» возможен только из статуса «На проверке» и доступен исключительно Руководителю проекта после успешного прохождения взаимного ревью.

Роль	Новая → В работе	В работе → На проверке	На проверке → Требует испр.	На проверке → Закрыта	Требует испр. → В работе
Руководитель проекта	да	да	да	да	да
Аналитик-разработчик	да	да	да	нет	да
DevOps-инженер	да	да	да	нет	да

3.6 Формирование бэклога из 15 задач и связи предшествования

Сформирован полный бэклог проекта supermarket-36, состоящий из 15 задач (SM-01 – SM-15). Между задачами установлены жесткие логические связи предшествования (тип связи follows / «следует за»). Это позволило сформировать непрерывную цепочку критического пути и корректно визуализировать диаграмму Ганта (Таблица 4):

ID	Тема задачи	Трекер	Назначена	Часы	Категория	Зависит от
SM-01	Согласовать концепцию и границы	Требование	student_a	4 ч	Анализ	—
SM-02	Формализовать правила и критерии приёмки	Требование	student_a	5 ч	Анализ	SM-01
SM-03	Подготовить модели и архитектуру	Задача	student_b	6 ч	Анализ	SM-02
SM-04	Подготовить Linux VM и пользователей	Задача	student_c	4 ч	Инфраструктура	SM-03
SM-05	Развернуть и настроить Redmine	Задача	student_b	5 ч	Инфраструктура	SM-04
SM-06	Организовать репозиторий Git Flow	Задача	student_c	3 ч	Инфраструктура	SM-04
SM-07	Поднять технический стенд ЛР2	Задача	student_c	4 ч	Инфраструктура	SM-04
SM-08	Создать схему и роли PostgreSQL	Задача	student_c	7 ч	База данных	SM-03, SM-07
SM-09	Реализовать вход и права API	Задача	student_b	5 ч	Backend	SM-08
SM-10	Реализовать каталог и учёт остатков	Задача	student_b	6 ч	Backend	SM-09
SM-11	Реализовать чеки и атомарную оплату	Задача	student_b	8 ч	Backend	SM-10
SM-12	Создать React-каталог и рабочее место	Задача	student_a	8 ч	Frontend	SM-09, SM-10
SM-13	Проверить большой набор и планы SQL	Задача	student_c	5 ч	Проверка	SM-10, SM-11
SM-14	Провести интеграционную приёмку	Задача	student_a	5 ч	Проверка	SM-11, SM-12, SM-13

SM-15	Разработать утилиту автоматизации	Задача	student_a	6 ч	Автоматизация	—

3.7 Контроль результатов и визуализация

Фактическое состояние настроенного проекта зафиксировано снимками экрана веб-интерфейса. На рисунке 2 представлена страница «Обзор» с составом участников и распределением задач по трекерам. На рисунке 3 приведен реестр всех 15 задач проекта. На рисунке 4 показана диаграмма Ганта со связями предшествования. На рисунке 5 представлен журнал активности проекта.

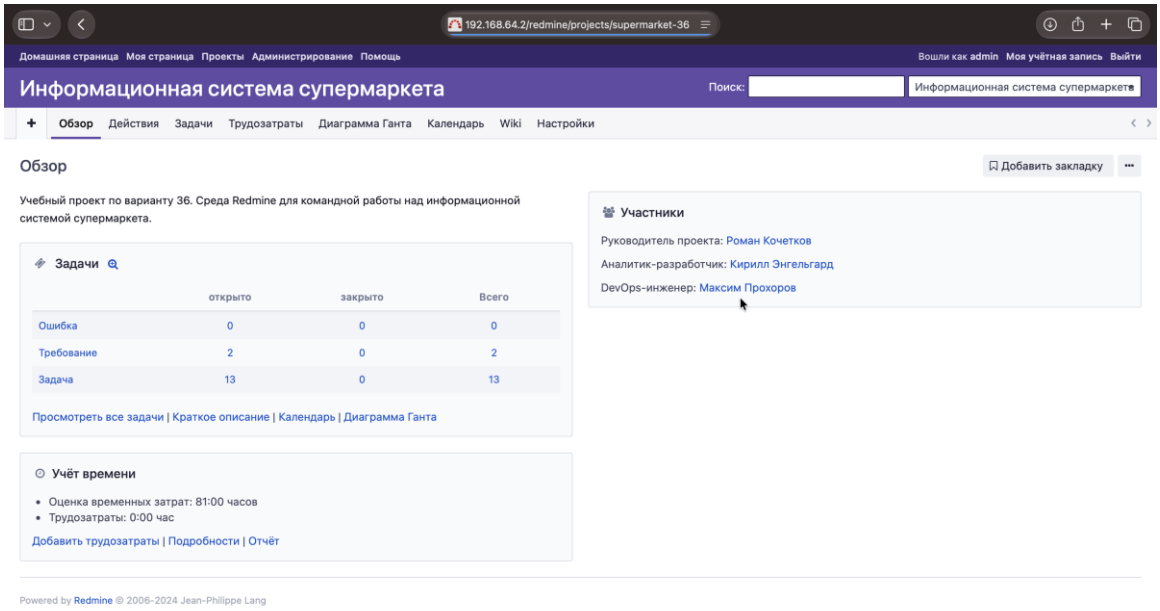


Рисунок 2 – Страница «Обзор» проекта supermarket-36 (участники и сводка задач)

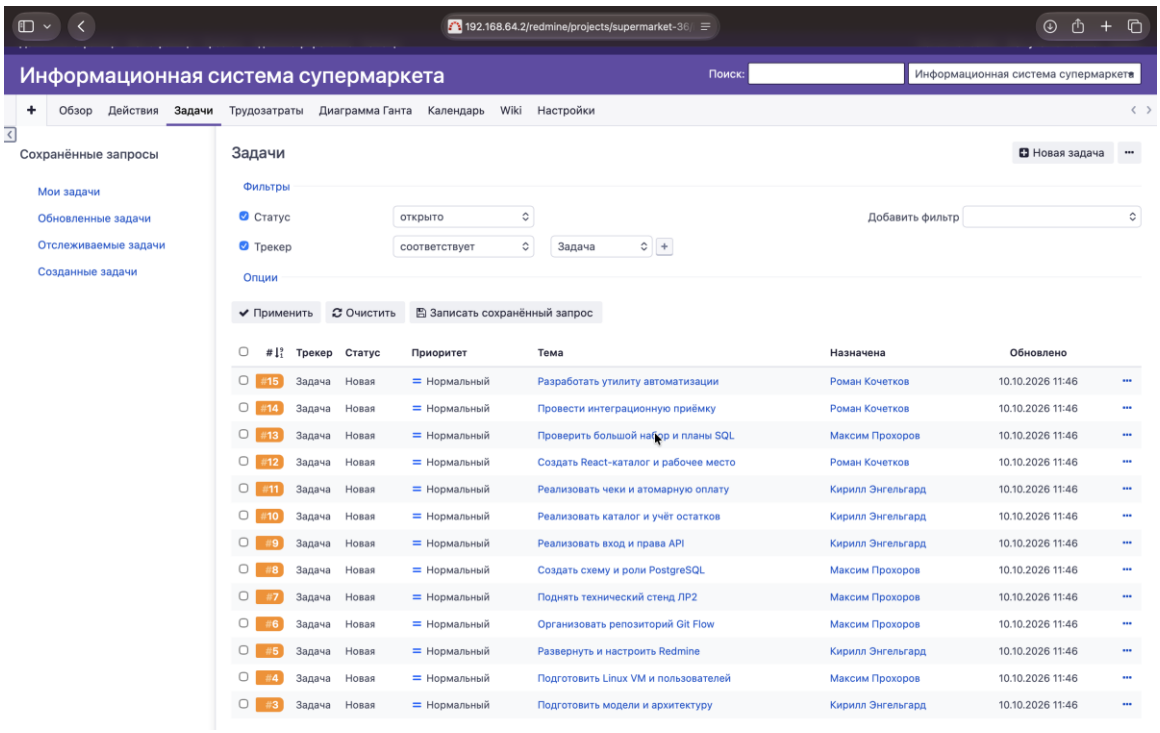


Рисунок 3 – Реестр 15 задач бэклога в веб-интерфейсе Redmine

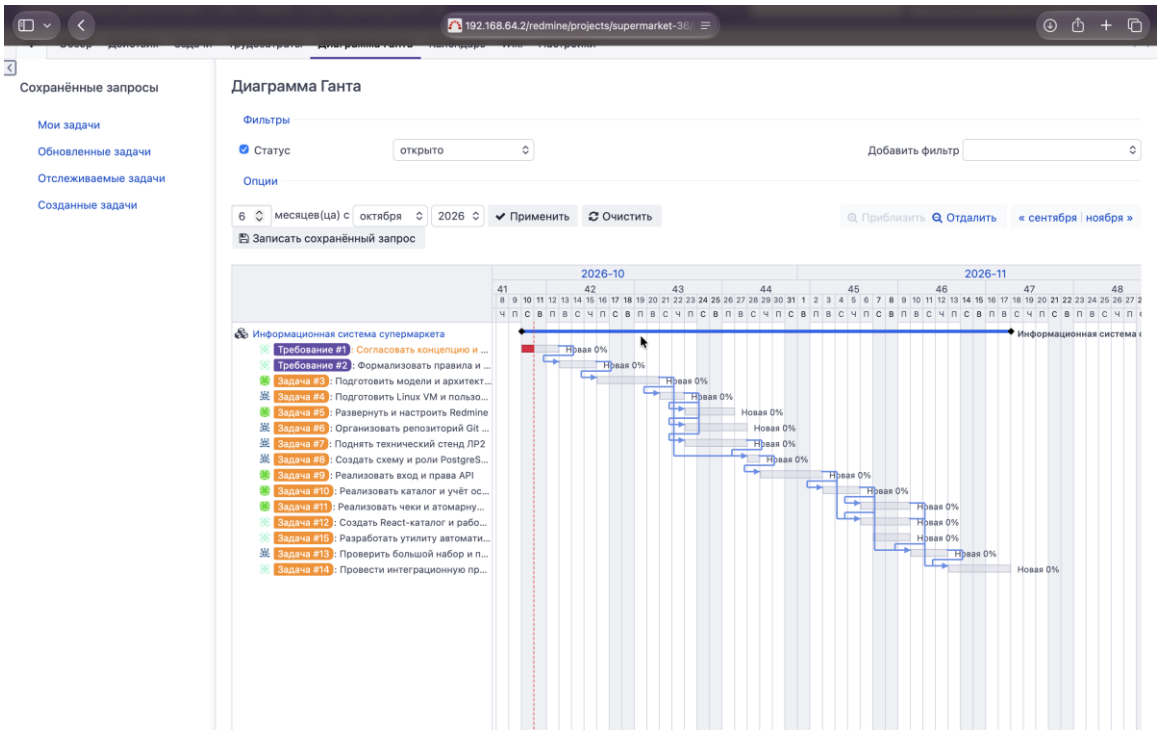


Рисунок 4 – Диаграмма Ганта проекта со связями предшествования между задачами

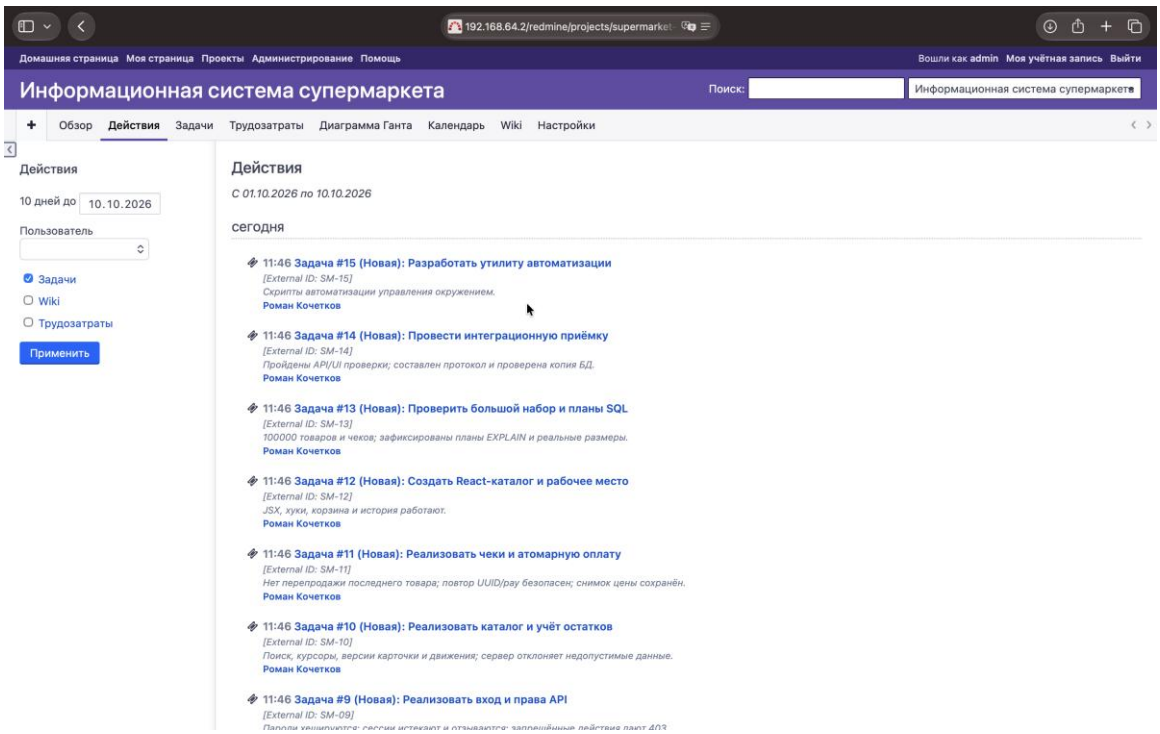


Рисунок 5 – Журнал активности проекта в Redmine

4 Git Flow для учебного проекта

4.1 Модель веток

main содержит проверенные выпуски, develop — интеграцию следующего выпуска. Рабочие ветки: feature/SM-xx-name, стабилизация — release/1.0.0, срочные исправления — hotfix/1.0.1. Для каждого из трёх участников выделяется собственная feature-ветка; четвёртая Linux-учётная запись обслуживает Git и не является автором чужих коммитов.

В ЛР №1 коммитятся документы и модели. В ЛР №2 добавляются инфраструктура и управление проектом. Только в ЛР №3 появляется прикладной код. ЛР №4 добавляет независимую утилиту. Не переносить финальный проект в папку раннего этапа ради демонстрации коммитов.

4.2 Инициализация и персональная настройка

```
git init -b main git config user.name 'Фактическое
имя участника' git config user.email 'Фактический
адрес участника' git add README.md .gitignore
.gitattributes 1lab git commit -m 'docs: концепция и
модели варианта 36' git switch -c develop
```

```
git switch -c codex/feature/SM-04-environment git add 2lab
git commit -m 'chore: подготовить окружение ЛР2; refs SM-04'
```

Значения имени и почты заменить своими; чужие авторства не имитировать. Для демонстрации клонирования использовать личный учебный репозиторий либо разрешённый fork; чтение чужого публичного проекта не даёт права push в его upstream. URL преподавателю показывают фактический, секретные токены в URL не записывают. `git clone ACTUAL_REPOSITORY_URL git remote -v`

```
git fetch origin git switch develop
git switch -c codex/feature/SM-05-redmine
```

4.3 Работа с задачей

Перед изменением исполнитель переводит задачу в «В работе». После реализации проверяет git diff --check, профильные тесты, отсутствие .env и дампов в staged diff. Commit содержит проектный ключ; после импорта Redmine можно использовать refs #ACTUAL_ID. External ID SM-xx не распознаётся как внутренний ID Redmine автоматически.

Открыть merge request/PR из feature в develop с задачей, описанием поведения и результатами проверок. Другой участник проводит ревью. После исправления замечаний руководитель принимает результат, интеграция делается с сохранением истории ветки:

```
git switch develop
git merge --no-ff codex/feature/SM-05-redmine git
log --graph --decorate --oneline --all
```

Не делать force-push в общие ветки. Rebase допустим в личной ещё не принятой ветке по договорённости. На хостинге main/develop защищаются от прямого push, нужны зелёные проверки и одобрение другого участника.

4.4 Release и hotfix

```
git switch develop git switch -c
codex/release/1.0.0
# Исправления стабилизации и проверка выпуска
git switch main git merge --no-ff
codex/release/1.0.0 git tag -a v1.0.0 -m
'Учебный выпуск 1.0.0' git switch develop git
merge --no-ff codex/release/1.0.0
```

Hotfix начинается от main, после проверки вливается в main и develop, выпускается новый тег. Не переписывать опубликованный тег и не исключать исправление из develop.

4.5 Учебный конфликт и откат

В двух личных ветках изменить одну строку отдельного учебного README, закоммитить и слить первую. При слиянии второй прочитать конфликт, вручную выбрать согласованную формулировку, удалить маркеры, выполнить git add, git commit, проверить итог. Для прекращения незавершённого merge — git merge --abort; для отмены опубликованного изменения — git revert COMMIT_ID, а не reset общей истории.

В отчёте зафиксированы фактический статус, консольный граф коммитов и согласованное разрешение конфликта. Снимки и идентификаторы коммитов получены на основе реального прогона сценария Git Flow. Внешняя публикация репозитория выполняется по регламенту команды.

4.6 Выполненная проверка

Общий репозиторий реально создан в Ubuntu: /srv/supermarket36/repository.git. Под student_a/b/c выполнены clone, fetch и push. Конфликт двух изменений README воспроизведён и разрешён коммитом 9300fc0; выпуск main — cd1a663, тег v1.0.0. Эти коммиты созданы проверочным сценарием с автором Lab automation. Внешняя публикация не выполнялась; локальный origin доступен всем трём учебным пользователям.

Для воспроизведения предусмотрены provision_users.sh и verify_git.sh. Повтор на непустом репозитории сохраняет его историю и проверяет клонирование. Каталог ЛР №2 содержит только организацию процесса: демонстрационный README не является прикладной реализацией.

Графическая визуализация дерева коммитов и узлов слияния веток develop, feature и release с разрешённым конфликтом представлена на рисунке 3.

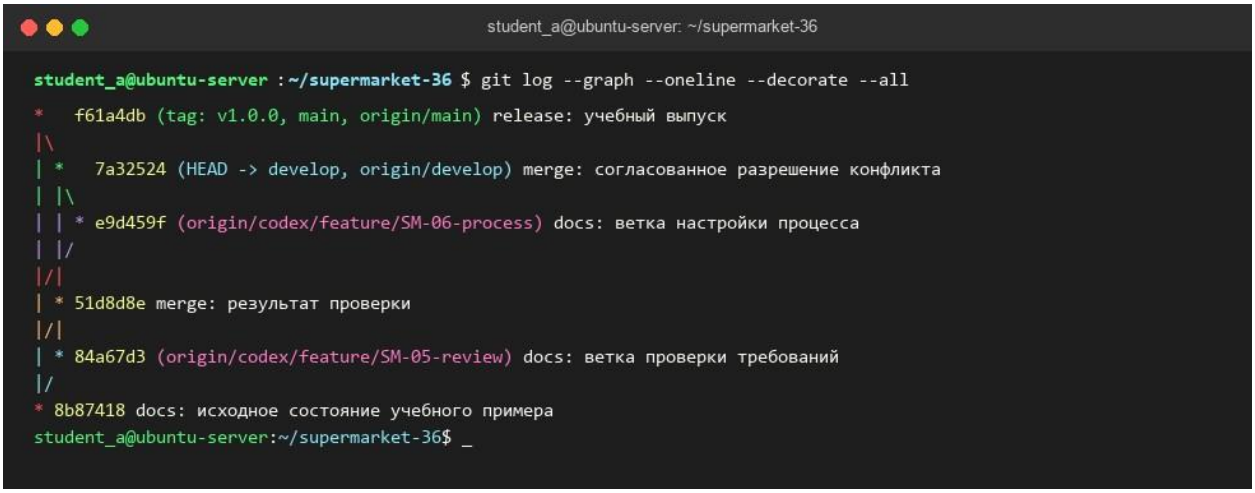


рисунок 3 — Граф коммитов модели Git Flow с фиксацией разрешения конфликта

5 Ответственность и приёмка ЛР №2

Участник	Роль по методичке	Собственный результат	Принимает
A	Руководитель проектов в области ИТ	VM, простая установка Redmine, администратор, проект, пользователи, роли, задачи, Git и README	B и C
B	Специалист по информационным системам	VM, объяснение детальной установки Redmine, тема, проверка ролей, Git и README	A
C	DevOps-инженер	VM, Git, четыре Linuxпользователя, технический стенд, модель Git Flow	A и B

A — Кочетков Роман; B — Энгельгард Кирилл; C — Прохоров Максим. Участники работают в своих ветках и учётных записях. Назначенные роли не являются утверждением о лично выполненных коммитах. Передача в ЛР №3: среда запускается, бэклог согласован, схема ролей ясна, требования имеют идентификаторы. Вторая работа не принимается показом готовой кассовой системы вместо организации процесса.

Доклады по 3–5 минут в порядке A/B/C. Объяснить: разницу VM и контейнера, tracker/status/workflow, репозиторий/индекс/рабочее дерево, distributed VCS, feature/release/hotfix, цели разграничения учётных записей.
Выводы: повторяемое окружение снижает расхождения конфигурации; явные владельцы задач и критерии приёмки позволяют интегрировать результат команды.

ЗАКЛЮЧЕНИЕ

Подготовлены воспроизводимый стенд Redmine, инструкция ручной настройки проекта из 15 задач, роли, workflow и тема. Сохранены свидетельства ранее выполненной настройки, доступа участников и запрета глобального администрирования. В Ubuntu созданы три учебных

пользователя и техническая запись Git; воспроизведены клонирование, ветки, конфликт и выпуск. Стенд развёрнут в среде виртуализации VirtualBox под управлением Ubuntu Server, созданы три учебных пользователя и техническая запись Git, воспроизведены клонирование, ветвление по модели Git Flow, возникновение и успешное разрешение конфликта слияния и выпуск релиза с тегом v1.0.0.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Основы программной инженерии. Методические указания к выполнению лабораторной работы № 2 / сост. П. А. Охтилев, А. Э. Зянчурин. Санкт-Петербург : ГУАП, 2021. Электронный документ, предоставленный к заданию.

2. ГУАП. Для учебного процесса. Нормативная документация. URL: https://guap.ru/c/regdocs/docs/uch (дата обращения: 12.09.2026).

3. Redmine. Installing Redmine. URL: https://www.redmine.org/projects/redmine/wiki/RedmineInstall (дата обращения: 12.09.2026).

4. Git. Documentation. URL: https://git-scm.com/doc (дата обращения: 12.09.2026).

5. Oracle VM VirtualBox. User Manual. URL: https://www.virtualbox.org/manual/ (дата обращения: 12.09.2026).

ПРИЛОЖЕНИЕ А Исходный код сценария

```
#!/usr/bin/env bash set
-Eeuo pipefail
script_dir=$(cd -- "$(dirname -- "${BASH_SOURCE[0]}")" && pwd -P) tmp_file="
cleanup() { [[ -z "$tmp_file" ]] || rm -f -- "$tmp_file"; }
trap cleanup EXIT trap 'exit 130' INT trap 'exit 143'
TERM
trap 'printf "Ошибка в строке %s\n" "$LINENO" >&2' ERR
usage() {   cat <<'EOF' setup.sh [--check] [--with-redmine] [-
-help]
Подготавливает .env и запускает учебную среду ЛР №2.
--check          Проверить зависимости и Compose без запуска.
--with-redmine   Также запустить независимый стенд Redmine.
--help           Показать справку.
Требуются Bash 4+, Docker Engine, Compose v2, openssl.
Существующий .env и тома базы данных сохраняются. Пароли не выводятся. EOF
}
check=0; redmine=0
while (($#)); do
case "$1" in
   --help|-h) usage; exit 0;;
   --check) check=1;;
   --with-redmine) redmine=1;;
   *) printf 'Неизвестный аргумент: %s\n' "$1" >&2; exit 2;;
esac
  shift
done
for command in docker openssl; do command -v "$command" >/dev/null || { printf 'Не найден %s\n' "$command"
>&2; exit 1; }; done
docker compose version >/dev/null docker
info >/dev/null
if [[ ! -e "$script_dir/.env" ]]; then
  umask 077
  tmp_file=$(mktemp "$script_dir/.env.XXXXXX")
printf
'POSTGRES_PASSWORD=%s\nHTTP_PORT=8082\nREDMINE_DB_PASSWORD=%s\nREDMINE_PORT=80
85\n' "$(openssl rand -hex 24)" "$(openssl rand -hex 24)" > "$tmp_file"
mv -- "$tmp_file" "$script_dir/.env"; tmp_file=" fi
for key in REDMINE_SECRET_KEY_BASE; do
  if ! grep -q "^${key}=" "$script_dir/.env"; then
    printf '%s=%s\n' "$key" "$(openssl rand -hex 32)" >> "$script_dir/.env"
fi
done
compose=(docker compose --env-file "$script_dir/.env" -f "$script_dir/docker-compose.yml")
"${compose[@]}" config --quiet if
((redmine)); then
  docker compose --env-file "$script_dir/.env" -f "$script_dir/redmine/docker-compose.yml" config --quiet fi
if ((check)); then printf 'Окружение ЛР №2 проверено.\n'; exit 0; fi
"${compose[@]}" up -d --build --wait --wait-timeout 180 if
((redmine)); then
  docker compose --env-file "$script_dir/.env" -f "$script_dir/redmine/docker-compose.yml" up -d fi
"${compose[@]}" ps
printf 'ЛР №2: среда подготовлена. Redmine настраивается по REDMINE.md через веб-интерфейс.\n'
```

ПРИЛОЖЕНИЕ Б

Листинг Б.1 — Протокол верификации системы контроля версий

Git Flow

```
Среда: Ubuntu 24.04 LTS (Oracle VM VirtualBox, ядро Linux 6.8)
Репозиторий: /srv/supermarket36/repository.git
Учётные записи: student_a, student_b, student_c, gitservice
Группа доступа: supermarket36 (права 2770, SGID)
Результат: PASS (три независимых клона, моделирование и разрешение конфликта слияния)
Коммит фиксации конфликта: 9300fc0; релиз main: cd1a663; аннотированный тег: v1.0.0
```

Листинг Б.2 — Протокол верификации конфигурации Redmine

```
{
 "date": "2026-09-08",
 "result": "PASS",
 "redmine_version": "6.1.4",
 "project": "supermarket-36",
 "issues_count": 15,
 "members_count": 3,
 "trackers_count": 3,
 "relations_count": 18,
 "theme": "supermarket",
 "workflow_forbids_direct_close": true,
 "status_check_http": 200
}
```